

Agent IA

1. Introduction au... Agents IA
2. Prompt Engineering
3. Architectures d'un Agent Moderne
4. MCP – Model Context... Protocol
5. Démonos : MCP sI PEST API Psviw AI Agent

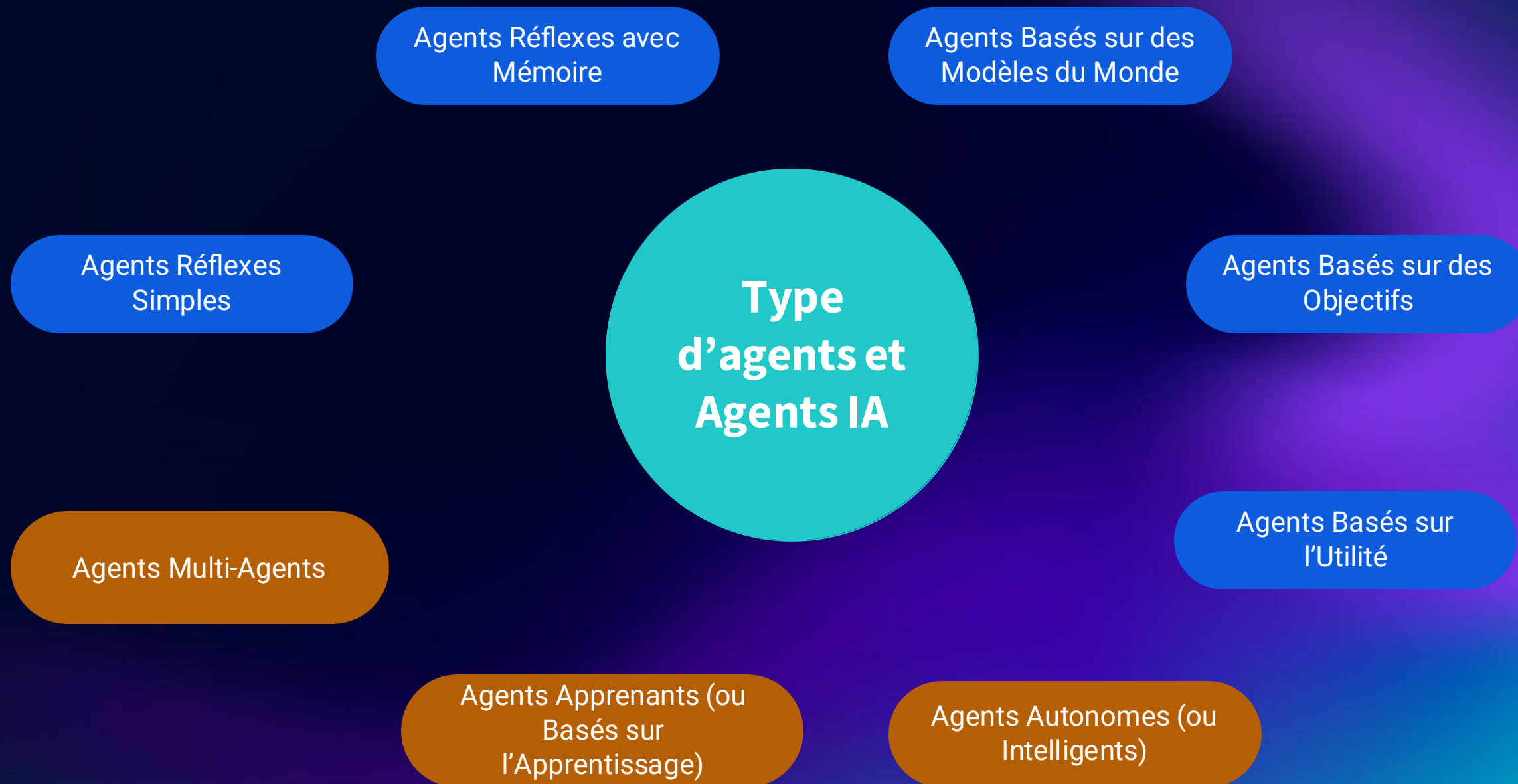
Qu'est-ce qu'un agent IA ?

“Une entité logicielle capable de percevoir son environnement (entrées), de prendre des décisions de manière autonome via un raisonnement, et d'agir sur l'environnement (sorties) pour atteindre ses objectifs”

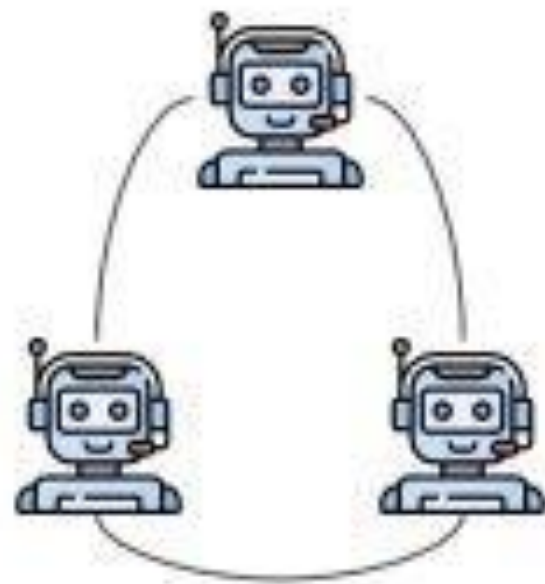
Wikipedia

Types d'Agent IA

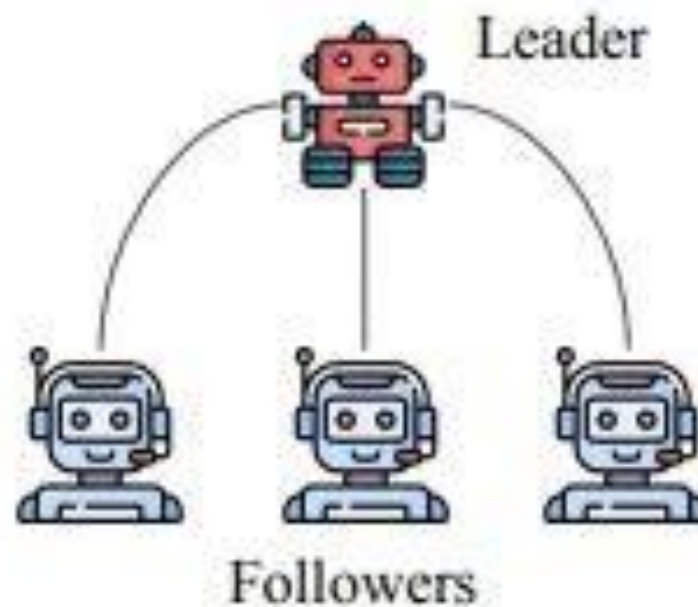
Types d'agents IA



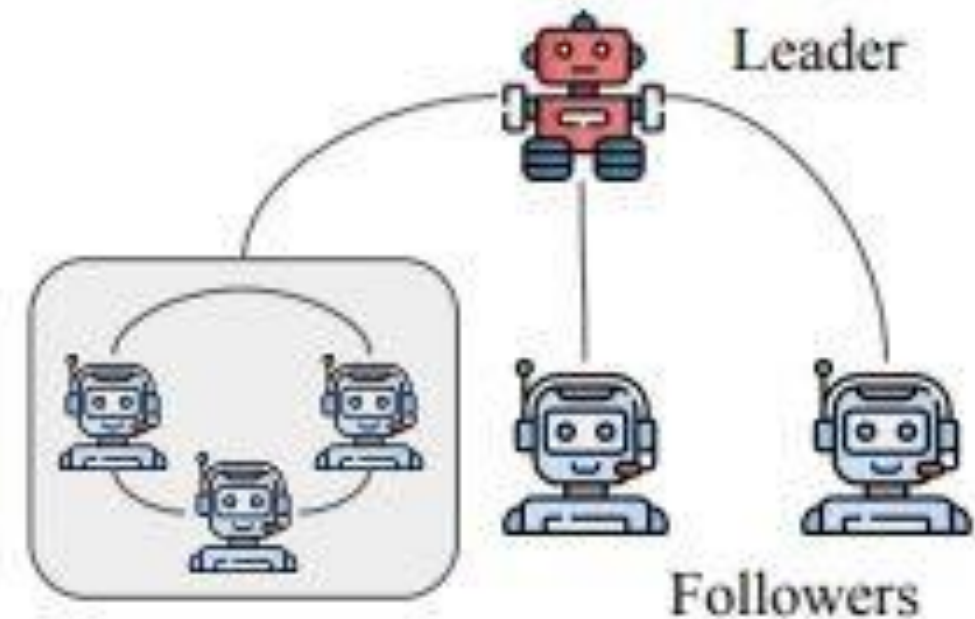
Types d'agents multiples (systèmes multi-agents)



a. Equi-level structure



b. Hierarchical structure



c. Nested structure

Challenges

- **Allocation optimale des tâches** : Assigner efficacement les sous-tâches aux agents en fonction de leurs compétences.
- **Raisonnement collaboratif** : Favoriser des débats constructifs entre agents pour affiner les résultats.
- **Gestion du contexte complexe** : Maintenir une compréhension cohérente des informations à différents niveaux (global, individuel, partagé).
- **Gestion avancée de la mémoire** : Développer des mécanismes robustes pour stocker, partager et sécuriser les informations entre agents.
- **Sécurité des Agents et la data leak**

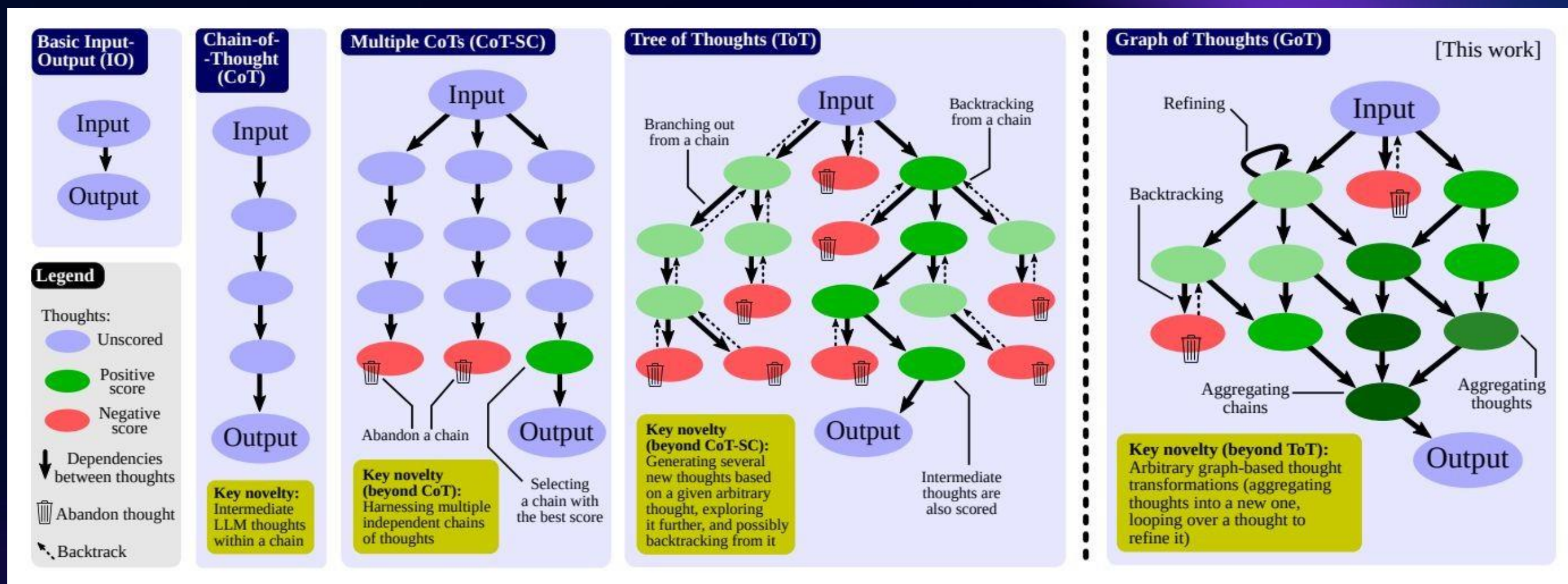
Prompt Engineering – Agent IA

LLM vs VLM vs VLA

Summary: LLM vs VLM vs VLA

Model Type	Processes	Examples	Use Cases
LLM (Large Language Model)	Text only	GPT-4, BERT, LLaMA	Chatbots, Content Generation, NLP
VLM (Vision-Language Model)	Images + Text	CLIP, BLIP, Flamingo	Image Captioning, Visual Q&A, Search
VLA (Vision-Language-Action)	Images + Text + Actions	PaLM-E, RT-2, SayCan	Robotics, AI Agents, Autonomous Systems

Prompt Engineering – Agent IA



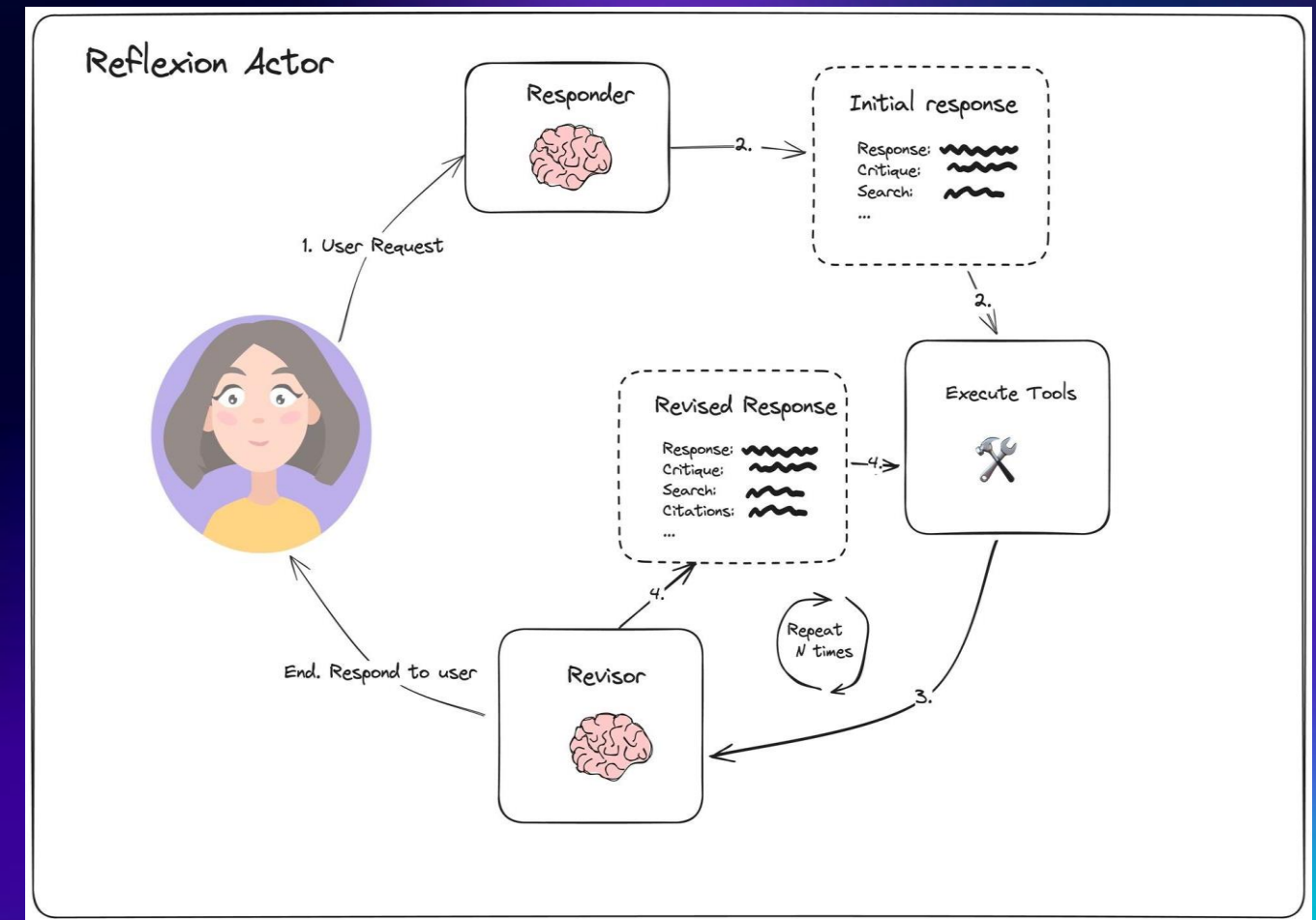
Pattern : Perception – Décision – Action

🧠 Réflexion (Self-Reflection)

L'agent analyse et critique sa propre réponse avant de la valider ou la corriger.

Permet d'identifier les erreurs et de corriger le raisonnement.

<https://github.com/langchain-ai/langgraph/blob/main/docs/docs/tutorials/reflexion/reflexion.ipynb>



Pattern : Planification

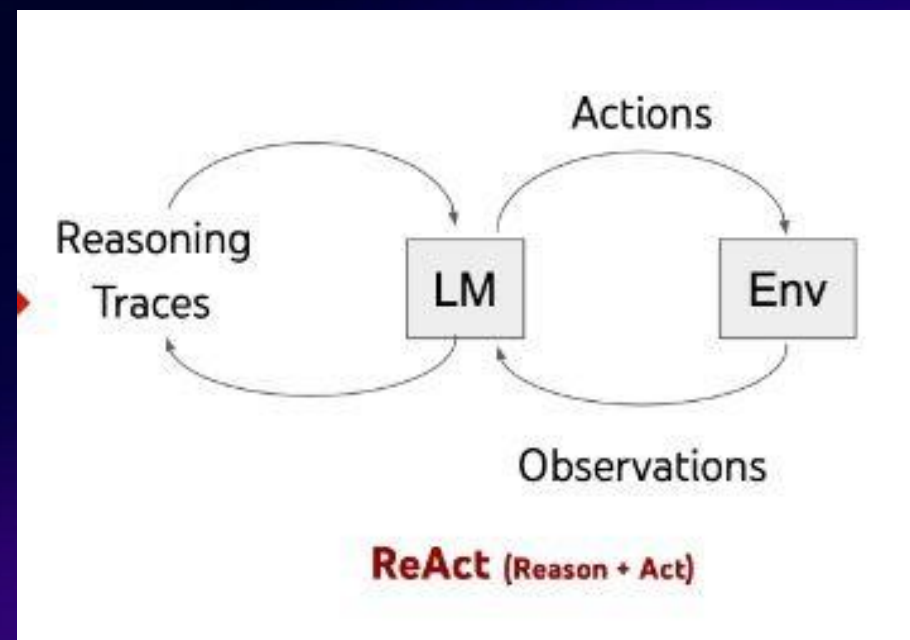
1. **Input** : "Quelle est la capitale de l'Australie ?"
2. **Answer**: "Sydney"
3. **Reflexion**: "Attends... est-ce correct ?"
 - a. "Hmm, non, Canberra est la capitale. Corrigeons."
4. **Final Answer**: "La capitale est Canberra."

Pattern : ReAct

🧠 ReAct : Reasoning + Acting

Une méthode pour doter les agents IA d'un raisonnement logique avec des actions dynamiques (<https://react-lm.github.io/>).

Boucle de pensée : tester → réfléchir → corriger → tester (en boucle) ⇒ action



Pattern : ReAct

1.  **Observation** : *"Quel temps fait-il à Marseille ?"*
2.  **Thought** : "Je dois connaître la météo à Marseille pour répondre."
3.  **Action** : Appeler l'API météo (MCP)
4.  **Observation** : *"Il fait 22°C et ensoleillé."*
5.  **Thought** : "Parfait, j'ai l'info. Je peux répondre."
6.  **Answer** : "Il fait 22°C et beau temps à Marseille."

Pattern : ReAct

Tu es un assistant intelligent qui répond à des questions en suivant une méthode structurée :

1. Exprimer ta réflexion sous "Thought"
2. Expliquer l'action que tu ferais sous "Action"
3. Donner le résultat obtenu sous "Observation"
4. Répondre clairement à la question sous "Answer"

Exemple :

Question: Quelle est la météo à Paris aujourd'hui ?

Thought: Je dois chercher la météo actuelle.

Action: weatherAPI("Paris")

Observation: Il fait 15°C et ensoleillé.

Answer: Il fait 15°C et ensoleillé à Paris aujourd'hui.

Maintenant, réponds à la question suivante :

Question: [ta question ici]



Réponse 1

Finished thinking.

Démarrage

Branche 1



Je préfère cette réponse

Score 1



Réponse 2

Finished thinking.

Démarrage

Branche 2



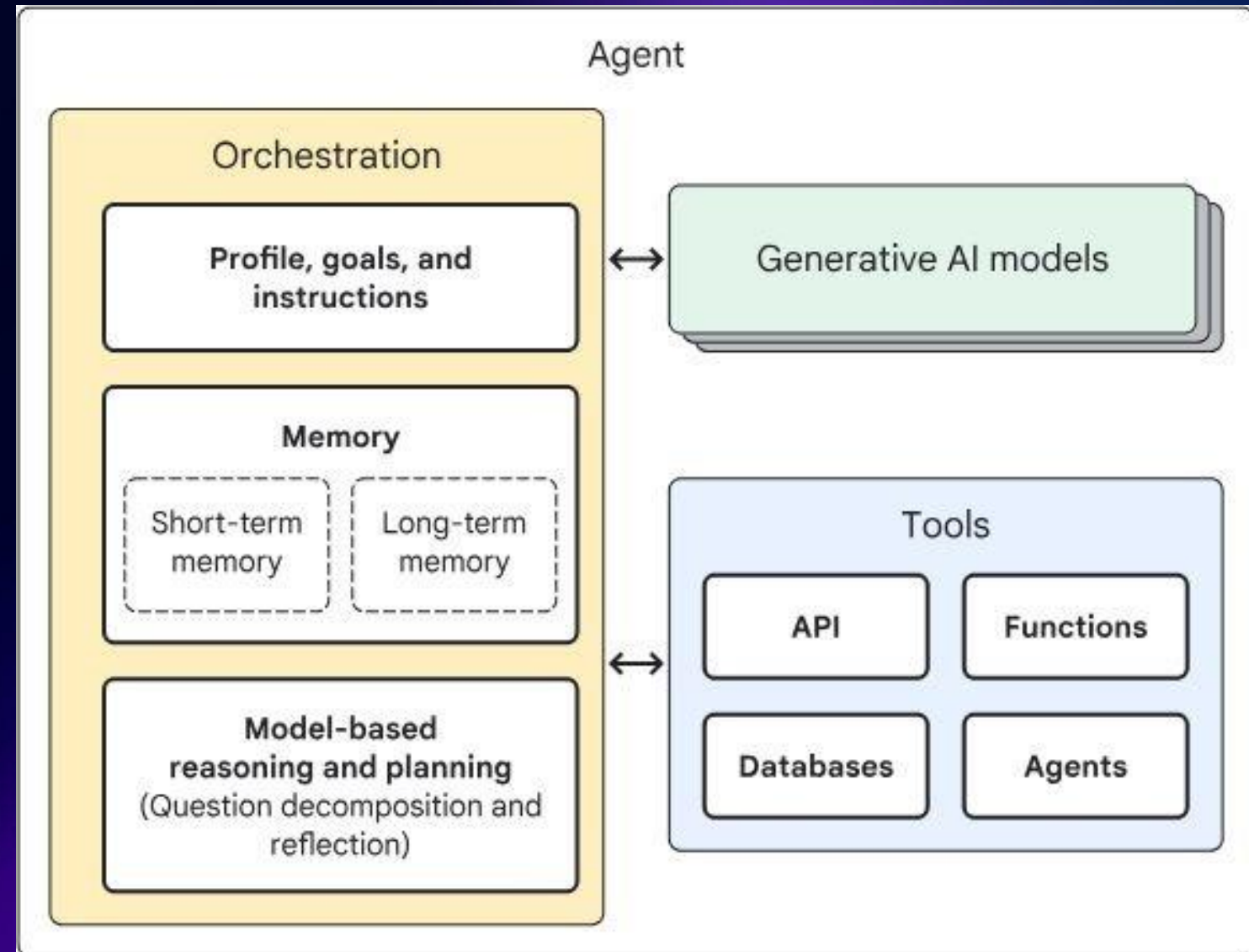
Je préfère cette réponse

Score 2

Agent IA – Modèles

Orchestration, Mémoire et Outils

- 🤖 **Orchestration :**
 - Multi agent / Leader / Follower
- **Mémoire :**
 - Focus sur le contexte
 - Réapprendre
- 💻 **Modèle**
 - LLM/VLM - ReAct ...
 - Prompt
- 🛠️ **Tools :**
 - Web Search / API
 - Serveur/Client MCP



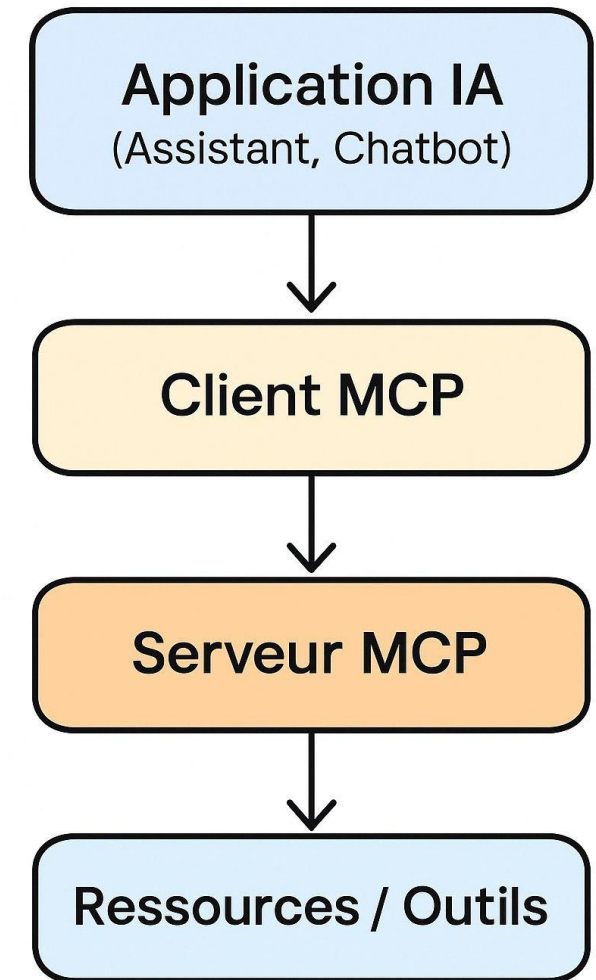
MCP

Model Context Protocol

Qu'est-ce que le Modèle Conceptuel de Protocole (MCP) ?

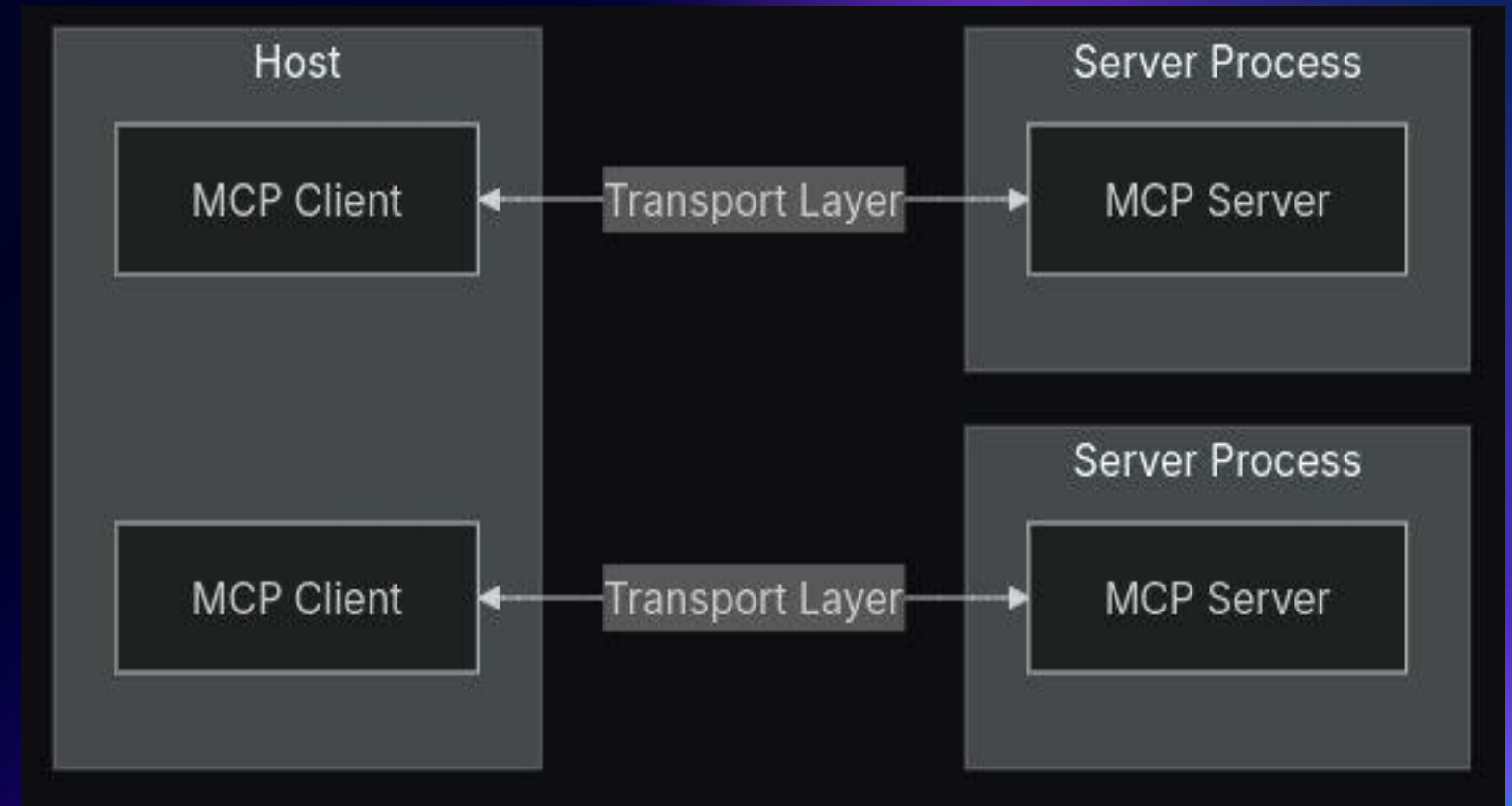
“MCP est un protocole ouvert qui standardise la manière dont les applications fournissent du contexte aux LLM, le protocol MCP offre un moyen standardisé de connecter les modèles d'IA à différents outils et sources de données”

- 🔌 Interface universelle
- 🧠 Connexion plug-and-play
- 🛠️ Interopérabilité native
- 🔄 Architecture modulaire
- 🌐 Accès local & distant



MCP : Architecture

- Architecture Client/Server(et Server/Server)
-  Les hôtes
 - Claud AI / OpenAI / N8N
-  Les clients
 - Client MCP
-  Les serveurs
 - Le serveur MCP



MCP : Concepts

Resources

Data à partager avec le host :

Logs, Fichiers, Images

Prompts

Prompts prédéfinis proposés par le serveur pour guider les LLMs. (COT)

Tools

Actions exécutables
API, SQL executor ...

Sampling

Demander un complément et validation d'information
(human-in-the-loop)

Roots

Permet au client de fournir un Contexte borné.

Exemple :

- Dataset / client
- Projet git / sous module

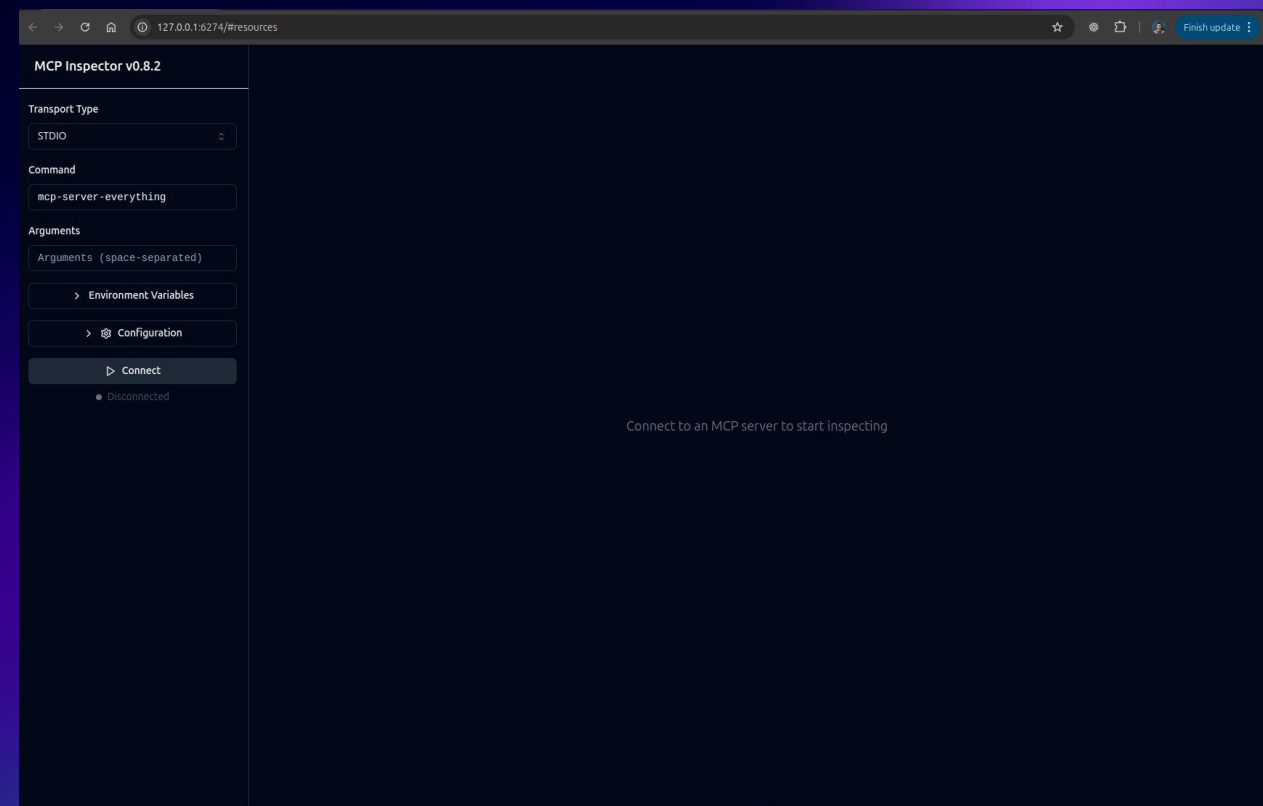
MCP : Simple Server

MCP Code Server avec Spring boot AI

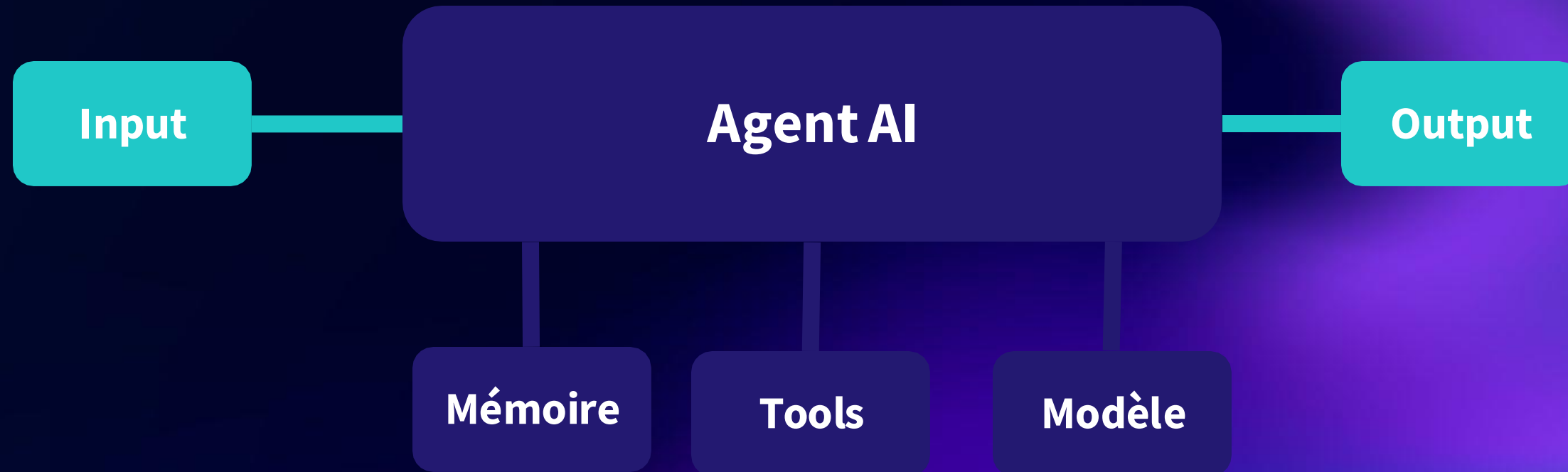
<https://github.com/aidevelopercode/ai-agents/tree/main/ai-agent-mcp-java-spring-boot>

MCP Client Inspector

`npx @modelcontextprotocol/inspector`



Agent IA



Workflow: Multi Agent



**Workflow = NoCods sI
LowCods**

NoCode Agents IA Builder

zapier

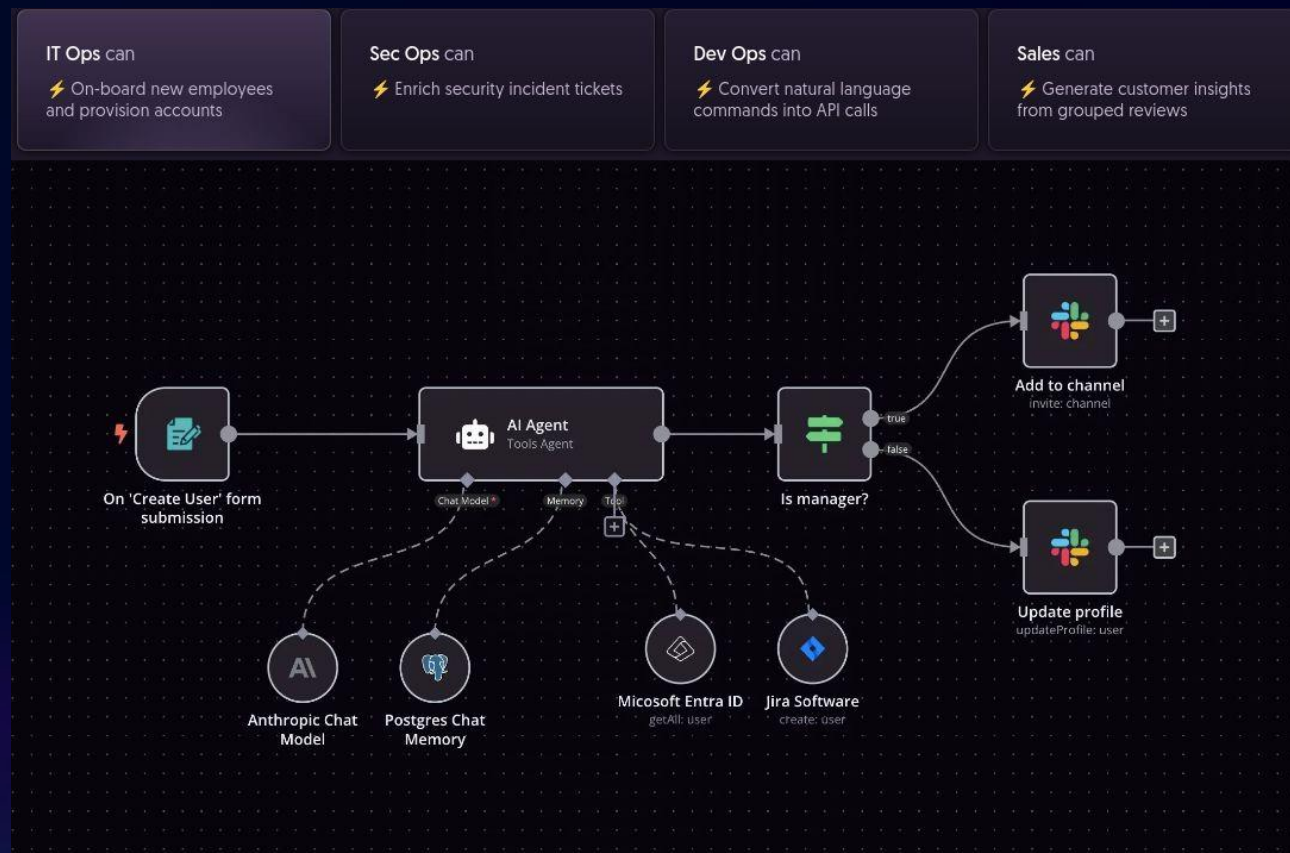


 **make**
formerly Integromat

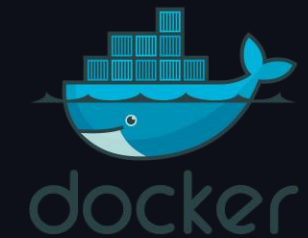
N8N Agents Ia Builder

<https://n8n.io/>

Libre d'utilisation en privé

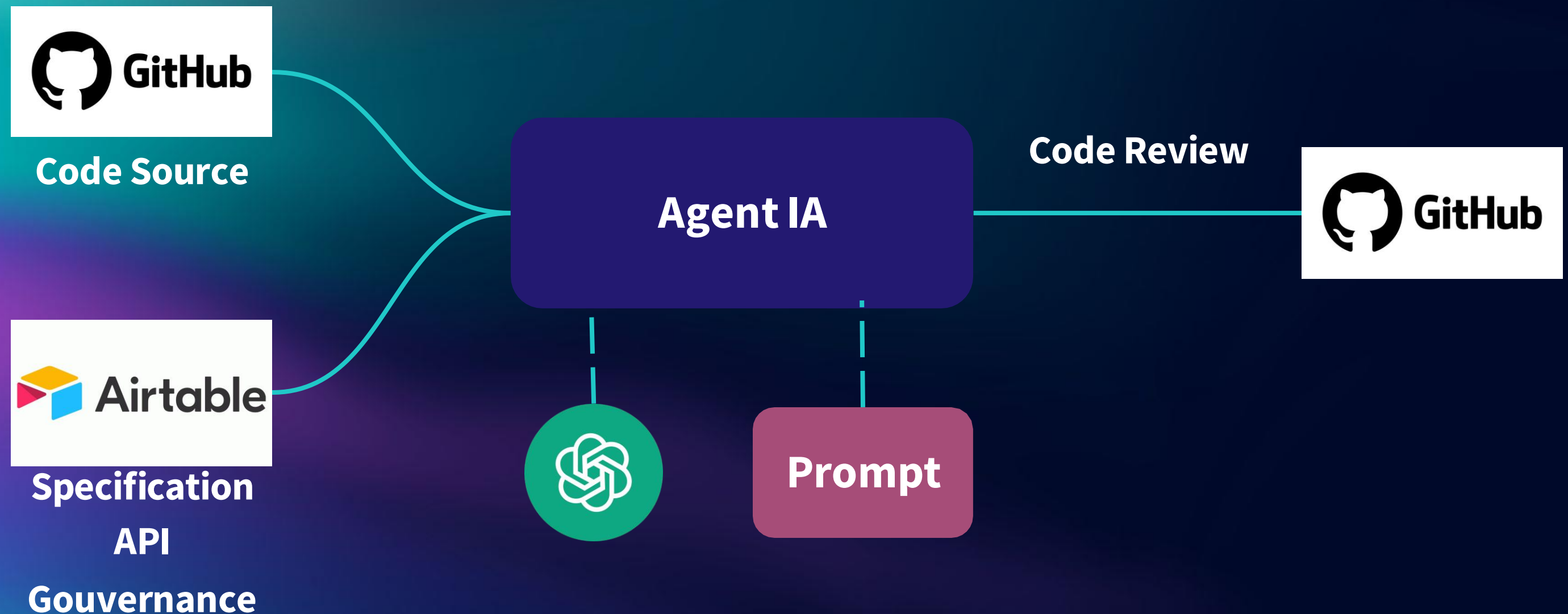


```
1  version: '3.8'
2  services:
3    n8n:
4      image: n8nio/n8n
5      container_name: n8n
6      restart: always
7      ports:
8        - "5678:5678"
9      volumes:
10       - ./n8n_data:/data
```



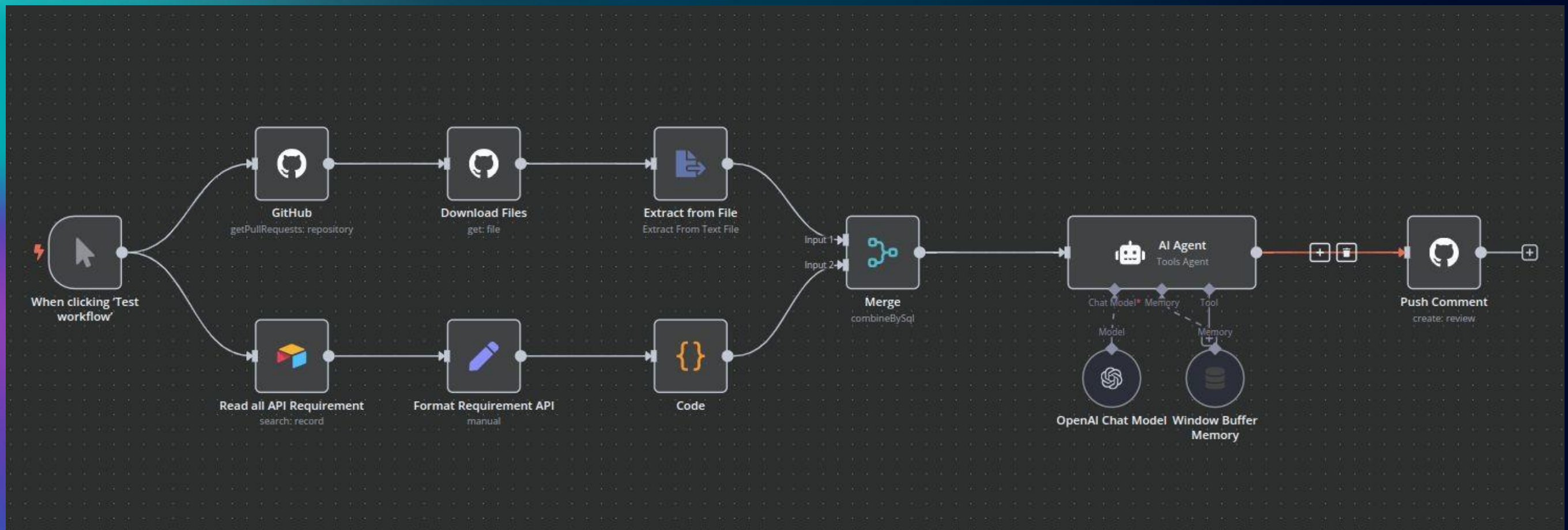
DEMO

Agent IA : API Governance Code Reviews



Agent IA : API Governance, Code Reviews

<https://aidevelopercode.app.n8n.cloud/workflow/DcEqMVlhbKMaM4mW>



Q&A

**Framswork sl
libs**

Agsnl IA